Section 6 Programming Challenge Problems

For this setion's challenge problems, we'll extend the *Kaleidoscope.py* and *HighCard.py* apps. For sample answers to these challenges, go to http://TeachYourKidsToCode.com.

#1: Random Sides and Thickness

Add more randomness to *Kaleidoscope.py* by adding two more random variables. Add a variable `sides` for the number of sides, then use that variable to change the angle we turn each time in the spiral loop (and therefore, the number of sides in the spiral) by using `360/sides + 1` as your angle instead of `91`. Next, create a variable called `thick` that will store a random number between 1 and 6 for the turtle pen's thickness. Add the line `t.width(thick)` in the right place to change the thickness of the lines of each spiral in our random kaleidoscope.

#2: Realistic Mirrored Spirals

If you know some geometry, two more tweaks make this kaleidoscope even more realistic. First, keep track of the direction (between 0 and 360 degrees) the turtle is pointing before drawing the first spiral by getting the result of `t.heading()` and storing it in a variable called `angle`. Then, before drawing each mirrored spiral, change the angle to the correct mirrored direction by pointing the turtle with `t.setheading()`. Hint: the second angle will be `180 - angle`, the third spiral's angle will be `angle - 180`, and the fourth will be `360 - angle`.

Then, try turning left after each drawn line for the first and third spirals, and right each time for the second and fourth spirals. If you implement these improvements, your kaleidoscope should really look like the spirals are mirror images of each other, in size, shape, color, thickness, and orientation. If you like, you can even keep the shapes from overlapping so much by changing the range of the x- and y-

values to `random.randrange(size,turtle.window_width()//2)` and `random.randrange(size,turtle.window_height()//2)`.

#3: War

Turn *HighCard.py* into the full game of War by making three changes. First, keep score: create two variables to keep track of how many hands the computer won and how many the user won. Second, simulate playing one full deck of cards by dealing 26 hands (perhaps using a `for` loop instead of our `while` loop, or keeping track of the number of hands played so far), then declaring a winner based on which player has more points. Third, handle ties by remembering how many ties have happened in a row; then, the next time one of the players wins, add the number of recent ties to that winner's score, and set the number of ties back to zero for the next round.

For sample answers to these programming challenges, go to http://www.TeachYourKidsToCode.com